Medical Computer Vision Lab

Python Coding Standards

Maintainer: Ashery Mbilinyi

Version 1.0 : October 16, 2026

Contents

Abstract **2**

1 Scope and Principles **3**
1.1 Scope . . . 3
1.2 Guiding Principles . . . 3

2 Code Layout and Formatting (PEP 8) **3**
2.1 Indentation . . . 3
2.2 Line Length . . . 3
2.3 Blank Lines . . . 3
2.4 Imports . . . 3
2.5 Whitespace . . . 4

3 Naming Conventions **4**

4 Comments and Documentation **5**
4.1 Comments . . . 5
4.2 Inline Comments . . . 5
4.3 Docstrings (PEP 257) . . . 5

5 Programming Recommendations (Pythonic Code) **6**
5.1 Comparisons . . . 6
5.2 Boolean Values . . . 6
5.3 String Formatting . . . 6
5.4 Exceptions . . . 6
5.5 Context Managers . . . 7

6 Tools to Enforce Standards **7**
6.1 Recommended Tooling . . . 7
6.2 Minimum Project Requirements . . . 7

7 References **7**

Abstract

This document defines the Python coding standards for **Medical Computer Vision (MCV) Lab**. We follow **PEP 8** as the primary style guide and emphasize **readability, reproducibility, and maintainability**. These standards apply to all research code, prototypes intended for reuse, and production-level systems developed in the lab.

1 Scope and Principles

1.1 Scope

These standards apply to:

- Research prototypes that will be shared within the group
- Code released with papers (GitHub repositories)
- Internal tooling and shared utilities (datasets, evaluation, plotting)
- Any code intended to be maintained beyond the original author

1.2 Guiding Principles

We follow the **Zen of Python** (`import this`) as a guiding philosophy:

- *Readability counts*
- *Simple is better than complex*
- *Explicit is better than implicit*

2 Code Layout and Formatting (PEP 8)

Python coding standards are primarily defined by **PEP 8**, the official style guide. Adhering to these standards ensures code is maintainable and easily understandable by other developers.

2.1 Indentation

- Use **4 spaces** per indentation level.
- Do **not** use tabs.

2.2 Line Length

- Limit all lines to a maximum of **79 characters**.
- When lines become too long, prefer:
  - breaking long expressions into multiple lines
  - using parentheses for implicit line continuation
  - splitting long strings into smaller parts

2.3 Blank Lines

- Separate top-level functions and class definitions with **two blank lines**.
- Separate method definitions inside a class with **one blank line**.

2.4 Imports

- Imports should be on **separate lines**.
- Group imports in the following order:
  1. Standard library

2. Third-party libraries
3. Local application / library-specific modules

- Separate groups using a **single blank line**.

Example:

```
# Standard library
from pathlib import Path
from typing import Optional

# Third-party
import numpy as np
import torch

# Local
from labproject.data import load_dataset
from labproject.models import build_model
```

2.5 Whitespace

- Avoid extraneous whitespace immediately inside parentheses/brackets/braces.
- Avoid whitespace before commas, semicolons, or colons.

Preferred:

```
x = (a + b) * c
items = [1, 2, 3]
my_dict = {"a": 1, "b": 2}
```

Avoid:

```
x = ( a + b ) * c
items = [1 , 2 , 3]
my_dict = { "a" : 1 , "b" : 2 }
```

3 Naming Conventions

- **Variables and functions:** `snake_case`
- **Classes:** `PascalCase`
- **Constants:** `UPPER_CASE_WITH_UNDERSCORES`
- **Modules and packages:** short, lowercase names (underscores allowed if needed)
- **Internal/private usage:** prefix with _ (leading underscore)

Example:

```
MAX_EPOCHS = 50

class TraumaClassifier(torch.nn.Module):
    def __init__(self, in_channels: int):
        super().__init__()
        self._in_channels = in_channels
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x

def train_one_epoch(model, dataloader):
    for batch in dataloader:
        pass
```

4 Comments and Documentation

4.1 Comments

- Comments should be **complete sentences**.
- Begin comments with # followed by a **single space**.

4.2 Inline Comments

- Use inline comments sparingly.
- Separate inline comments by at least **two spaces**.

Example:

```
step += 1  # Increment global step counter
```

4.3 Docstrings (PEP 257)

- Use `"""triple double quotes"""`.
- Add docstrings to all public modules, classes, functions, and methods.
- Docstrings must clearly describe:
  - purpose
  - input arguments
  - return values
  - major assumptions / side effects

Example:

```
def load_ct_volume(path: str) -> np.ndarray:
    """Load a CT volume from disk.

    Args:
        path: Path to a NIfTI file.

    Returns:
        The CT volume as a NumPy array.
    """
    return np.load(path)
```

5 Programming Recommendations (Pythonic Code)

5.1 Comparisons

- Use `is None` / `is not None` for `None` checks.

Preferred:

```
if x is None:
    return
```

Avoid:

```
if x == None:
    return
```

5.2 Boolean Values

- Do not compare booleans to `True` or `False` with ==.

Preferred:

```
if greeting:
    print("Hello")
```

Avoid:

```
if greeting == True:
    print("Hello")
```

5.3 String Formatting

- Prefer **f-strings** for readable formatting.

```
name = "Ashery"
score = 0.92
print(f"{name} achieved AUROC={score:.3f}")
```

5.4 Exceptions

- Avoid bare `except:`.
- Catch specific exceptions.

Preferred:

```
try:
    value = int(x)
except ValueError:
    value = 0
```

5.5 Context Managers

- Use `with` statements for resource management (files, etc.).

```
from pathlib import Path

path = Path("results.txt")
with path.open("w", encoding="utf-8") as f:
    f.write("done\n")
```

6 Tools to Enforce Standards

To ensure consistency across the lab, we rely on automated tools.

6.1 Recommended Tooling

- **Formatter:** `black` (preferred)
- **Linting:** `flake8` or `pylint`
- **Import sorting:** `isort`
- **Type checking:** `mypy`

6.2 Minimum Project Requirements

Every shared lab project repository should include:

- `README.md` with setup + usage
- A reproducible environment file (e.g., `requirements.txt` or `environment.yml`)
- Tool configuration (e.g., `pyproject.toml` for `black`/`isort`)

7 References

- PEP 8 — Style Guide for Python Code: https://peps.python.org/pep-0008/
- PEP 257 — Docstring Conventions: https://peps.python.org/pep-0257/
- Zen of Python: https://peps.python.org/pep-0020/
- Black: https://black.readthedocs.io/
- Flake8: https://flake8.pycqa.org/
- isort: https://pycqa.github.io/isort/
- mypy: https://mypy.readthedocs.io/